

Mini Project *

EENG350: Systems Exploration, Engineering, and Design Laboratory

Hisham Sager and Prachi Sharma

Department of Electrical Engineering
Colorado School of Mines

Spring 2026

1 Introduction

This project is intended to help you start working together as a team. This means having you team discuss and agree to your team norms, setting up development areas like GitHub and collaboration tools, learning how to define project task that are modular, with clear and limited intersections with other task, and learning how to debug an integrated system. To focus the work, you have the following assignment:

Integrate your two wheel system and computer vision system so that the computer vision can act as a user interface to define set-points for the wheels. Specifically, you will place two pieces of tape on each wheel, 180 degrees apart. The two pieces are differentiated writing a 0 on one tape, and 1 on the other. When the system is turned on, both wheels are set so that 0 is facing up. Using a marker that can be detected by the camera, the wheels are turned according to the following logic:

Marker visible in image quadrant	Left Wheel	Right Wheel
NE	0	0
NW	0	1
SW	1	1
SE	1	0

The controller that turns the wheels is able to reject disturbances, so that if you attempt to turn the wheel by hand, the controller will pull the wheel back to the correct position (i.e. integral control). This pi should display an updated picture of the camera image and indicate the current position of the marker. You will also display the current status of the wheels using the LCD. The display should be something like the following:

```
Goal Position: 0 1
```

The numbers change depending on the marker position.

* Developed and edited by Tyrone Vincent and Vibhuti Dave with assistance from Darren McSweeney and Paul Sampson. This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-nc-sa/3.0/> or send a letter to Creative Commons, 171 Second Street, Suite 300, San Francisco, California, 94105, USA.

2 Demonstration Expectations and Grading

Behavior	Points
Marker quadrant identified	5
Displays goal position on LCD display with no lag	5
One motor can turn to a desired position	5
Both motors can turn to an independent desired position	5
Integral control is implemented	5
Goal position is communicated by PI to Arduino	5
Total	30 points

3 Documentation Expectations and Grading

Your documentation will consist of the material placed in your GitHub repository. The GitHub repository should have a readme.md that describes the purpose of the repository and its organization. Your code for the miniproject should be in the repository in a folder named Mini Project, with it's own readme.md, and further file organization that makes it easy to find the different types of code. It should contain well documented code that implements the full project, as well as supplemental Python, Arduino or Simulink/Matlab code you can use to verify the operation of different subsystems (i.e. code to check that the communication is working, or code to verify that the motor controller is working that doesn't depend on the rest of the system operating.) For example, you should be able to quickly simulate either an open loop (voltage command) or closed loop (position command with PI controller implemented) step response, perform an experimental step response, and quickly compare the results in Matlab.

Item	Points
Repository contains mini project code	5
Code is well documented, with headers that describe code purpose and any hardware connections that are required	5
Repository is well documented and easy to navigate	5
Supplementary code exists that facilitates debugging, including Simulink models of the control system	5
Total	20 points

4 Setting up a Github repository on the Pi

1. If you don't already have an account, sign up for a free account at github.com.
2. Create a new repository. From your home page/dashboard, look for the + symbol in the upper right. Choose an appropriate name, and you can make the repository private. Initialize the repository with a README file, and choose python for the .gitignore file. You can choose a license if you wish.
3. When the repository is created, edit the readme and indicate that this is a repository for your seed lab class. As you create new folders within the repository, re-edit this file to keep the description up to date
4. Add you team members to this repository. From the repository main page, on the upper right, click on settings. Then, from the menu on the left hand side, click on Collaborators.
5. Create an access token for this repository. Follow the directions [here](#). You can select access as required, but recommend Access:Read and write for "Administration", "Commit Statuses", "Contents" and "Pull requests". **When the token is generated, copy it to a place where you can access it.** You won't be able to see it on Github again!
6. Go to your Pi and open a terminal. Go to the home page of your github repository and click the green Code button. Copy the HTTPS link to your repository. On the pi, type

```
git clone https://github.com/<repository>
```

where `https://github.com/<repository>` is the link that you copied above. When asked for your user name, type it in. When asked for the password, enter in the token that you generated above.¹

7. If the repository is cloned successfully, you should find a folder on your Pi that is the name of your repository. At the terminal go to this directory using `cd` and the name of the folder. Type

```
mkdir MiniProject
cd MiniProject
touch test.txt
```

This created an empty file in this folder called `test.txt`. Type

```
git add .
git status
```

As shown by `git status`, `git add .` added the file to your local repository. You can commit this change using

```
git commit -m "Added the file test.txt"
```

(Git may ask you to set your account's identity - follow the directions if so.) The text in quotes is a comment that describes what the commit is about. This has updated your local repository with the change, but now you want to update the repository on Github. You can do that with a [push](#) command

```
git push
```

If you refresh the browser on the Github website, you should see this new folder and file in the repository.

By using the Github repository, you have a way of tracking your development, and you are using version control that will allow you to go back to a previously working version if something goes wrong. You can also have multiple people working with clones of the repository at the same time by using [branching](#). Updates to the Github repository can be brought back to the pi using [pull](#). There are a variety of guides and tutorials that you can access to use Github to its fullest potential, including [this one](#).

5 Use Threading to Speed Up Processing

The LCD you are using on the Pi takes a while to update, and the Adafruit library provided will wait for it to finish updating. Unfortunately, that means that your program will wait for the LCD too if it is running on the same thread. Fortunately, there is a module called `threading` which will allow you to create a thread running parallel to your main thread, which will take over the job of waiting on the LCD.

A threading thread is started in the following way:

```
import threading

def myFunction(myArgs):
    print("Doing something.")

myThread = threading.Thread(target=myFunction, args=(myArgs,))
myThread.start()
# Carry on doing other things while the thread runs...
```

For regularly recurring events, you might put a loop in your thread and make use of the `queue` module to pass variables to it:

¹If you are copying the password from elsewhere, note that the paste command on the terminal program for the raspberry Pi is `shift+ctrl+v`. You can see this in the edit menu on the terminal program.

```

import queue
import threading
from random import random
from time import sleep

q = queue.Queue()

def myFunction():
    # *****
    # Initialize your LCD here
    # *****

    while True:
        if not q.empty():
            gotSomething = q.get()
            print("I got: {}".format(gotSomething))
            # *****
            # Write new data to the LCD here
            # *****

myThread = threading.Thread(target=myFunction, args=())
myThread.start()

while True:
    # Do some things...
    putSomething = random()

    # Send it to the thread
    if True:      # Put your own conditional here (i.e. ArUco marker moved)
        q.put(putSomething)

    # Carry on...
    sleep(1) # I wouldn't keep this here outside of this example...

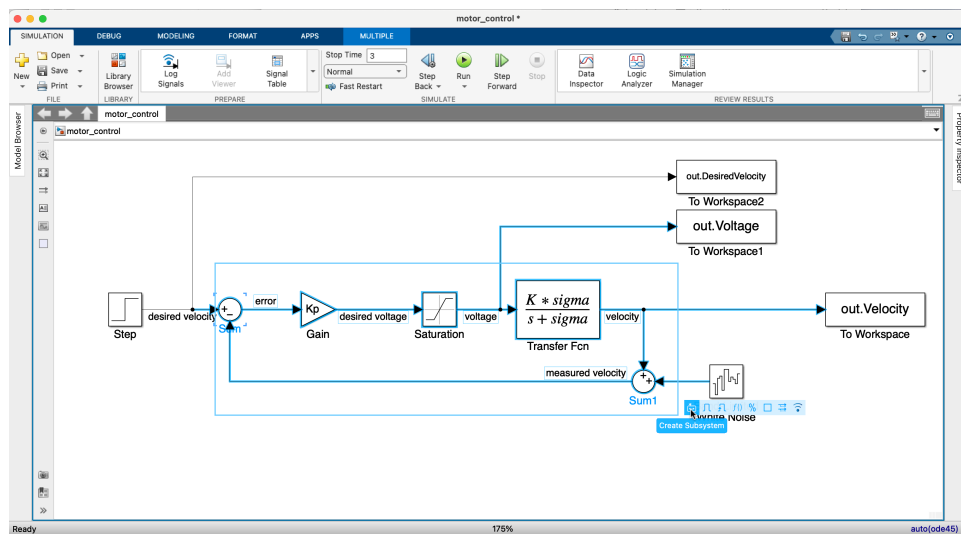
```

There are a few caveats to keep in mind when using threading. For one thing, once a thread object finishes, the `start()` function will raise an exception. You will need to re-initialize the thread in order to use it again. Additionally, when using `print()` in child threads, things can get a bit confusing, as two threads may try to print to stdout at the same time. If this will be a problem for you, consider making use of the logging module to help keep track of parallel tasks while debugging, referencing the documentation here:

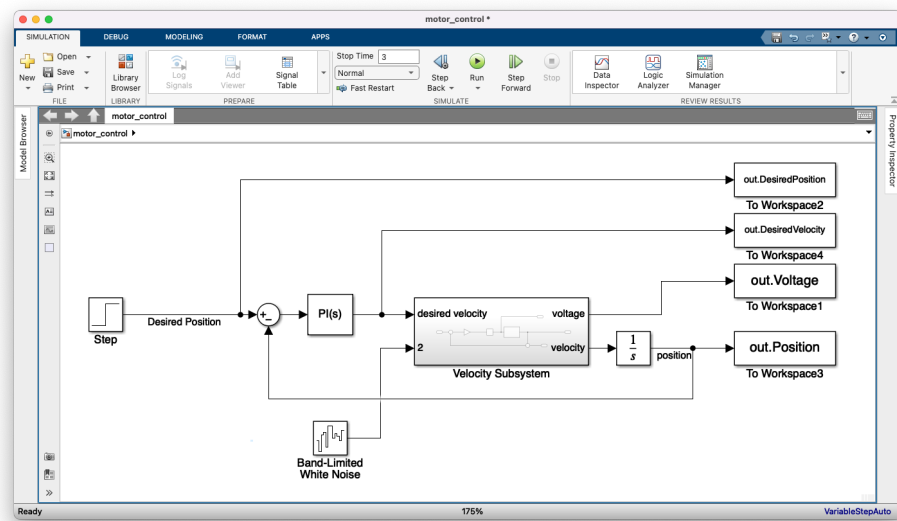
<https://docs.python.org/2/library/logging.html>

6 Position Control

Since you already have a working velocity controller, you can wrap another control loop around that to create a position controller. On your Simulink block diagram, select the blocks are shown below, and create a subsystem as shown in this figure:



Since position is the integral of velocity, we can add an integrator to calculate angular position. Also add a PID block, summing block, and re-arrange connectors to create the following block diagram.



Note that we have changed the name of two of the two workspace blocks and reconnected them to the position and desired position signals. Also note that we have selected PI as the controller in the PID block.

We see that the PI controller will take the error between the actual position and desired position as the input, and provide the desired velocity as its output. Your existing controller will then take that desired velocity command and compute the motor voltage.

You tune the gains of the PI controller by hand, or you can use the automated tuning available in the PID block. If you tune by hand, I would suggest starting with the integral gain equal to zero, and adjusting the proportional gain. You can use a similar script as in Assignment 2 to plot the results, with the appropriate changes in variable names.

The implementation of the controller will be similar to before, except that you will need to include an integration calculation. It is highly recommended that you read the EEGN307 lecture [PIDcontrol.article.pdf](#) that is available on Canvas. Also, since you will be implementing this on two wheels, it will be useful to define variables as two element arrays. The implementation will be of the following form.

```
float integral_error[2] = {0};
```

```

void loop()

:   code to define variables and get position and velocity. You will change some of your existing variables to two
:   element arrays, and add the other necessary variables.
:
for(i=0;i<2;i++) {
    pos_error[i] = desired_pos[i] - actual_pos[i];
    integral_error[i] = integral_error[i] + pos_error[i]*((float)desired_Ts_ms /1000);
    desired_speed[i] = Kp_pos * pos_error[i] + Ki_pos * integral_error[i];
    error[i] = desired_speed[i] - actual_speed[i];
    Voltage[i] = Kp*error[i];
}

:   rest of code to output the voltages and wait for loop time to finish.
:

```

You may also wish to add an anti-windup feature to handle integrator windup (see the PID control lecture for more details).